

Document Number: P2629R0

Date: 2022-07-08

Reply to: Gonzalo Brito Gadeschi <gonzalob_at_nvidia.com (<http://nvidia.com>)>

Authors: Gonzalo Brito Gadeschi

Audience: Concurrency

barrier token-less split arrive/wait

- barrier token-less split arrive/wait
 - Motivation
 - Tony tables
 - User guide
 - Teaching examples
 - Producer-consumer pipeline example
 - Rationale
 - Rationale for waiting without a token
 - Rationale for arriving without a token
 - Impact
 - Impact on users
 - Impact on implementations
 - Impact on/from other proposals
 - Unresolved questions
 - References
 - Proposed wording

Motivation

This proposal extends `std::barrier` with better support for data-pipelining. These pipelines are very common in, e.g., Deep Learning and HPC applications [0 (<https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>)].

In Deep Learning pipelines targetting modern NUMA systems, two groups of threads, “producers” and “consumers”, synchronize access to the shared resources of a pipeline stage using a pair of barriers. Consumer threads wait on the “consumer barrier”, process data once unblocked, and arrive on the “producer barrier” to signal that shared resources are safe to

reuse. Producer threads wait on the “producer barrier”, reuse shared resources to load the next data set, and arrive on the “consumer barrier” to signal consumer threads that this data set is ready for use.

In this synchronization pattern, consumer threads wait on the consumer barrier without arriving on it, and arrive on the producer barrier without waiting on it; and vice-versa for producer threads.

However, `std::barrier` APIs require threads to arrive on a barrier before waiting on it. This proposal allows producer and consumer threads to synchronize efficiently.

Tony tables

Real applications implement multi-stage pipelines with multiple shared resources. The following table shows a single stage of a consumer-producer pipeline involving one consumer thread, one producer thread, one `data` buffer as a shared resource, and a pair of `std::barriers`.

Note: the code of each thread runs in an infinite loop.

Before		After
Producer Thread	Consumer Thread	Producer Thread
<code>producer.arrive_and_wait();</code> <code>produce(data);</code> <code>consumer.arrive();</code>	<code>consumer.arrive_and_wait();</code> <code>consume(data);</code> <code>producer.arrive();</code>	<code>producer.wait_parity(pp);</code> <code>produce(data);</code> <code>consumer.arrive_and_discard()</code>

Before this proposal:

- **Producer thread:**
 1. Waits on the consumer thread to signal that `data` can be reused by arriving and waiting on the `producer` barrier.
 2. Produces `data`.
 3. Signals that `data` is ready by arriving at the `consumer` barrier; throws token away.
- **Consumer thread:**
 1. Waits on `data` becoming ready by arriving and waiting on the `consumer` barrier.
 2. Consumes `data`.

3. Signals the producer thread that it can reuse data by arriving on the producer barrier; throws token away.

These steps cause threads to unnecessarily:

1. **modify a barrier**: by calling `arrive_and_wait` when they only need to wait,
2. **stall** at the `arrive` until the `barrier_token` is constructed when they will immediately discard it afterwards, and
3. **cast `[[nodiscard]]` away**: to avoid the warning produced by discarding the token.

After this proposal:

- **Producer thread:**

1. Waits on the consumer thread to signal that data can be reused by ~~arriving and waiting on the producer barrier.~~
2. Produces data .
3. Signals that data is ready by arriving at the consumer barrier without producing a barrier token.; ~~throws token away.~~

- **Consumer thread:**

1. Waits on data being ready by ~~arriving and waiting on the consumer barrier.~~
2. Consumes data .
3. Signals the producer thread that it can reuse data by arriving on the producer barrier without producing a token. ~~throws token away.~~

This proposal addresses the three pre-existing issues raised above.

User guide

The `arrive_and_discard` API is semantically equivalent to `static_cast<void>(barrier.arrive())` but is easier to implement efficiently.

This section therefore focuses on how to use:

```
| void wait_parity(bool phase_parity) const;
```

The phase completion step of `std::barrier` advances the barrier to the “next phase” ([thread.barrier#class-1.3](http://eel.is/c++draft/thread.barrier#class-1.3) (<http://eel.is/c++draft/thread.barrier#class-1.3>)).

This proposal guarantees that:

- the initial barrier phase is “phase 0”, and
- the successor of “phase N” is “phase N+1”.

This proposal defines that the parity of:

- even barrier phases is `false`, i.e., the parity of “phase 0” is `false` (or 0);
- odd barrier phases is `true`, i.e., the parity of “phase 1” is `true` (or 1).

The `wait_parity(bool phase_parity)` API waits on the barrier phase parity changing from the specified `phase_parity` to a different parity. This is analogous to the `atomic::wait(value)` API, which waits on the atomic value changing from `value` to a different value.

Teaching examples

This example uses a barrier to print text in order:

```
void houston() {
    std::barrier barrier(2);                // A
    std::thread apollo13([&barrier] {
        std::cout << "Okay, Houston, we've had a problem here" << std::endl;
        static_cast<void>(barrier.arrive()); // B
    });
    bool phase_parity = false;
    barrier.arrive_and_wait();              // C
    std::cout << "This is Houston. Say again, please." << std::endl;
    apollo13.join();
}
```

The initial phase of a freshly initialized `barrier` is 0 and its parity is `false`. To implement this example with the proposed APIs we can:

- Line A: use an `expected_count` of 1 instead of 2, since only 1 thread will arrive on each barrier phase.
- Line B: use `arrive_and_discard`, since this thread will not wait on the barrier.
- Line C: use `wait_parity(false)` to wait on the phase parity of initial barrier phase, which is “phase 0” and has the phase parity `false`, to change to `true` (i.e. advancing to “phase 1”):

```

void houston() {
    std::barrier barrier(1);           // A
    std::thread apollo13([&barrier] {
        std::cout << "Okay, Houston, we've had a problem here" << std::endl;
        barrier.arrive_and_discard(); // B
    });
    bool phase_parity = false;
    barrier.wait_parity();              // C
    std::cout << "This is Houston. Say again, please." << std::endl;
    apollo13.join();
}

```

The following example shows a thread that uses `wait_parity` to wait on all `barrier` phases by alternating the `phase_parity` it waits on, on every iteration:

```

bool phase_parity = false;
while(true) {
    barrier.wait_parity(phase_parity);
    phase_parity = !phase_parity;
    other_barrier.arrive_and_discard();
}

```

This thread arrives on some `other_barrier`, to make sure that other threads arriving on `barrier` don't cause it to "skip" waiting on a particular phase.

Producer-consumer pipeline example

The following code snippet (try it out in [compiler-explorer](https://godbolt.org/z/W95Y1q1vY) (<https://godbolt.org/z/W95Y1q1vY>)) shows a full working example of the producer-consumer pipeline described above and showcases how to combine these APIs:

```

#include <thread>
#include <barrier>

int data = -1;
constexpr int max_count = 5;

std::barrier producer(1);
std::barrier consumer(1);

void produce() {
    int count = 0;
    // Producer thread tracks the barrier phase separately.
    // On the first iteration, the producer thread waits
    // on "phase 1" of the barrier, which is an odd phase:
    bool phase = true;
    do {
        data = count++;
        // Once the data is produced, it notifies the consumer
        // by arriving on the "consumer" barrier using
        // "arrive_and_discard", because this thread does not
        // need to wait on that barrier:
        consumer.arrive_and_discard()
        if (count == max_count) return;
        producer.wait_parity(phase);
        // After unblocking from the barrier, the phase to
        // be waited on in the next iteration is updated:
        phase = !phase;
    } while(true);
}

void consume() {
    int count = 0;
    bool phase = true;
    while(true) {
        consumer.wait_parity(phase);
        phase = !phase;
        assert(data == count++);
        producer.arrive_and_discard();
        if (count == max_count) return;
    }
}

int main() {
    std::thread t(produce);
    consume();
    t.join();
    return 0;
}

```

Rationale

Applications cannot reuse `std::barrier` to solve these problems, and instead must re-implement it to add these APIs.

This proposal chooses to extend `std::barrier` with two APIs to enable applications to re-use it.

Rationale for waiting without a token

The proposed `wait_parity` API is a replacement for `arrive_and_wait` in certain situations.

The `arrive_and_wait` API has significant latency:

- Sends a message to modify the memory to arrive at the barrier.
- Waits on the response with the old value, required to construct the `arrival_token` that the API returns.
- Finally calls `wait` to wait on the barrier phase to flip.

The `wait_parity` API enables applications that “know” the parity of the barrier phase they want to wait on, and that are able to do so safely, to just wait on the barrier, without having to arrive on it first.

Rationale for arriving without a token

The `arrive_and_discard` API is a replacement for `arrive` in situations where the calling thread is not going to wait at the barrier.

The `arrive` API:

- Sends a message to modify the memory to arrive at the barrier.
- Waits on the response with the old value, required to construct the `arrival_token` that the API returns.

The roundtrip latency of sending the message and waiting for the response is significant in large NUMA architectures.

P2588 () “`barrier`’s phase completion guarantees” proposes relaxing the specification of `barrier` to enable implementations to complete the phase in one of the threads that `waits` at the barrier by requiring that at least one thread waits for the phase to flip.

On an implementation that completes the phase on a thread that waits, and on a proper hardware architecture with support for far atomics, `arrive_and_discard` is fire-and-forget. It sends a message to modify the barrier count, but it does not need to wait for a response.

During the call to `wait`, the implementation then picks one thread, and uses it to complete the phase, unblocking all other threads when the phase completes.

The reference implementation of this proposal on compiler-explorer (<https://godbolt.org/z/W95Y1q1vY>) provides such an implementation.

On large NUMA architectures and data-pipelining applications, the consumer barrier can be placed close to the consumer threads, and the producer barrier can be placed close to the producer threads.

This enables threads to poll the barrier locally and benefit from HW acceleration to do so, but increases the latency of arriving at the barrier.

The `arrive_and_discard` API brings the latency to zero on such an implementation, because the thread does not need to wait for a response to be able to make progress.

Impact

Impact on users

None, the semantics of existing code do not change, and it does not change the ABI.

Impact on implementations

The `arrive_and_discard` API imposes minimal overhead on implementations, since it can be naively implemented as:

```
void arrive_and_discard(ptrdiff_t update = 1) {  
    static_cast<void>(arrive(update));  
}
```

The implementation impact of `wait_parity` is “to be determined”. In the reference implementation, it is implemented naively as:

```
void wait_parity(bool phase_parity) const {  
    wait(arrival_token { .phase = phase_parity });  
}
```


Impact on/from other proposals

P2588 () proposes relaxing `std::barrier` 's phase completion guarantees. Depending on the outcome of this proposal, `wait_parity` would, just like `wait`, be able to complete the barrier's phase. In that case, it might make sense to not make `wait_parity` a `const` member function.

Unresolved questions

- Do we need the “diff” in the barrier constructor to state that it initializes the barrier to the 0-th phase? That is already addressed in `thread.barrier.class.1`.
- Impact on existing implementations, to be resolved by prototyping the feature in `libc++` and `libstdc++`.

References

- [0]: See “Asynchronous Execution” section in: The NVIDIA Hopper Architecture In-Depth (<https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>).
- P2588 (): `barrier` 's phase completion guarantees.
- Reference implementation of this proposal and P2588 () on `compiler-explorer` (<https://godbolt.org/z/W95Y1q1vY>).

Proposed wording

`thread.barrier.class` (<http://eel.is/c++draft/thread.barrier.class>):

```

namespace std {
    template<class CompletionFunction = see below>
    class barrier {
    public:
        using arrival_token = see below;

        static constexpr ptrdiff_t max() noexcept;

        constexpr explicit barrier(ptrdiff_t expected,
                                   CompletionFunction f = CompletionFunction());
        ~barrier();

        barrier(const barrier&) = delete;
        barrier& operator=(const barrier&) = delete;

        [[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);
void arrive and discard(ptrdiff_t update = 1);
        void wait(arrival_token&& arrival) const;
void wait_parity(bool phase_parity) const;

        void arrive_and_wait();
        void arrive_and_drop();

    private:
        CompletionFunction completion;      // exposition only
    };
}

```

1. The initial phase of a barrier is the 0-th phase. The successor of the n-th barrier phase is the (n+1)-th phase.
2. Each barrier phase consists of the following steps:
 1. The expected count is decremented by each call to `arrive` or `arrive_and_drop`.
 2. When the expected count reaches zero, the phase completion step is run. For the specialization with the default value of the `CompletionFunction` template parameter, the completion step is run as part of the call to `arrive` or `arrive_and_drop` that caused the expected count to reach zero. For other specializations, the completion step is run on one of the threads that arrived at the barrier during the phase.
 3. When the completion step finishes, the expected count is reset to what was specified by the `expected` argument to the constructor, possibly adjusted by calls to `arrive_and_drop`, and the ~~next~~successor of the current phase starts.
3. Each phase defines a phase synchronization point. Threads that arrive at the barrier during the phase can block on the phase synchronization point by calling `wait`, and will remain blocked until the phase completion step is run.

4. The phase completion step that is executed at the end of each phase has the following effects:

1. Invokes the completion function, equivalent to `completion()` .
2. Unblocks all threads that are blocked on the phase synchronization point.

The end of the completion step strongly happens before the returns from all calls that were unblocked by the completion step. For specializations that do not have the default value of the `CompletionFunction` template parameter, the behavior is undefined if any of the barrier object's member functions other than `wait` are called while the completion step is in progress.

5. Concurrent invocations of the member functions of `barrier`, other than its destructor, do not introduce data races. The member functions `arrive` and `arrive_and_drop` execute atomically.

6. `CompletionFunction` shall meet the `Cpp17MoveConstructible` (Table 30) and `Cpp17Destructible` (Table 34) requirements. `is_nothrow_invocable_v<CompletionFunction&>` shall be true.

7. The default value of the `CompletionFunction` template parameter is an unspecified type, such that, in addition to satisfying the requirements of `CompletionFunction` , it meets the `Cpp17DefaultConstructible` requirements (Table 29) and `completion()` has no effects.

8. `barrier::arrival_token` is an unspecified type, such that it meets the `Cpp17MoveConstructible` (Table 30), `Cpp17MoveAssignable` (Table 32), and `Cpp17Destructible` (Table 34) requirements.

```
static constexpr ptrdiff_t max() noexcept;
```

- Returns: The maximum expected count that the implementation supports.

```
constexpr explicit barrier(ptrdiff_t expected,  
                           CompletionFunction f = CompletionFunction());
```

- Preconditions: `expected >= 0` is true and `expected <= max()` is true .
- Effects: Sets both the initial expected count for each barrier phase and the current expected count for the first phase to `expected` . Initializes `completion` with `std::move(f)` . Starts the first phase which is the 0-th barrier phase.
 - [Note 1: If `expected` is 0 this object can only be destroyed. — end note]
- Throws: Any exception thrown by `CompletionFunction` 's move constructor.

```
[[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);
```

- Preconditions: `update > 0` is true, and `update` is less than or equal to the expected count for the current barrier phase.
- Effects: Constructs an object of type `arrival_token` that is associated with the phase synchronization point for the current phase. Then, decrements the expected count by `update`.
- Synchronization: The call to `arrive` strongly happens before the start of the phase completion step for the current phase.
- Returns: The constructed `arrival_token` object.
- Throws: `system_error` when an exception is required (`thread.req.exception` (<http://eel.is/c++draft/thread.req.exception>)).
- Error conditions: Any of the error conditions allowed for mutex types (`thread.mutex.requirements.mutex` (<http://eel.is/c++draft/thread.mutex.requirements.mutex>)).
 - [Note 2: This call can cause the completion step for the current phase to start. — end note]

```
void arrive_and_discard(ptrdiff_t update = 1);
```

- Effects: Equivalent to: `arrive(update)`.

```
void wait(arrival_token&& arrival) const;
```

- Preconditions: `arrival` is associated with the phase synchronization point for the current phase or the immediately preceding phase of the same barrier object.
- Effects: Blocks at the synchronization point associated with `std::move(arrival)` until the phase completion step of the synchronization point's phase is run.
 - [Note 3: If `arrival` is associated with the synchronization point for a previous phase, the call returns immediately. — end note]
- Throws: `system_error` when an exception is required (`thread.req.exception` (<http://eel.is/c++draft/thread.req.exception>)).
- Error conditions: Any of the error conditions allowed for mutex types (`thread.mutex.requirements.mutex` (<http://eel.is/c++draft/thread.mutex.requirements.mutex>)).

```
void wait_parity(bool phase_parity) const;
```

- Effects: Equivalent to: `wait(token)`, where `token` is associated with the synchronization point of the current phase if the parity of the phase ordinal matches `phase_parity`, or the previous phase otherwise.

UNRESOLVED QUESTION: if P2588 is accepted, we might not want to make `wait_parity` const.

```
void arrive_and_wait();
```

- Effects: Equivalent to: `wait(arrive())` .

```
void arrive_and_drop();
```

- Preconditions: The expected count for the current barrier phase is greater than zero.
- Effects: Decrements the initial expected count for all subsequent phases by one. Then decrements the expected count for the current phase by one.
- Synchronization: The call to `arrive_and_drop` strongly happens before the start of the phase completion step for the current phase.
- Throws: `system_error` when an exception is required (`thread.req.exception` (<http://eel.is/c++draft/thread.req.exception>)).
- Error conditions: Any of the error conditions allowed for mutex types (`thread.mutex.requirements.mutex` (<http://eel.is/c++draft/thread.mutex.requirements.mutex>)).
 - [Note 4: This call can cause the completion step for the current phase to start. — end note]